

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS), CHENNAI – 600 117

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

MINI PROJECT REPORT

IMAGE PROCESSING ON SKIN DISEASES USING DEEP LEARNING (DermaAI — AI Skin Image Classifier)

Submitted by

BHUVANESH B (24619140)

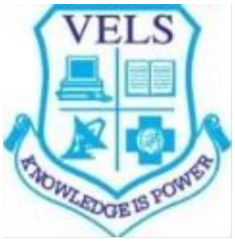
VASANTH P (24619135)

LOGA PRAKASH M (24619120)

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF ENGINEERING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS) • CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this mini project report entitled “IMAGE PROCESSING ON SKIN DISEASES USING DEEP LEARNING” is the bonafide work of BHUVANESH B (24619140), VASANTH P (24619135), and LOGA PRAKASH M (24619120), carried out under the guidance of Ms.SHANTHI(Assistant professs

Ms.SHANTHI(Assistant professor)

Department of Artificial Intelligence and Machine Learning, Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

Certified further that, to the best of our knowledge, the work reported in this project has been prepared for academic evaluation and has not been submitted elsewhere for the award of any degree or diploma.

SIGNATURE OF THE HEAD OF THE DEPARTMENT

Dr. K. Ulaga Priya
Head, Department of AI & ML
VISTAS, Chennai

SIGNATURE OF THE PROJECT GUIDE

Ms.SHANTHI(Assistant professor)
Project Guide, AI & ML
VISTAS, Chennai

Submitted for the Mini Project evaluation and Viva-Voce examination held on ..... at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the mini project report entitled “IMAGE PROCESSING ON SKIN DISEASES USING DEEP LEARNING” is based on the project work carried out by us for academic purposes at Vels Institute of Science, Technology and Advanced Studies (VISTAS).

We have documented the supplied DermaAI Streamlit program and project presentation faithfully. The proposed ResNet-50 and SVM methodology is distinguished from the actual Keras inference code where the supplied materials do not establish that they are the same implementation.

BHUVANESH B (24619140)	VASANTH P (24619135)	LOGA PRAKASH M (24619120)
Student Signature	Student Signature	Student Signature

ACKNOWLEDGEMENT

It is a great delight and privilege to acknowledge the support and encouragement received during the completion of this mini project. We are grateful to Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai, for providing the academic environment and facilities required to undertake this work.

We express our sincere gratitude to the management of VISTAS, the Head of the Department of Artificial Intelligence and Machine Learning, and our project guide, Ms.SHANTHI(Assistant professor) for their guidance, encouragement, and valuable suggestions. We also thank the faculty members, technical staff, and library staff for their support throughout the project.

Finally, we thank our parents and friends for their encouragement and support, which helped us complete the project and its documentation.

BHUVANESH B – 24619140

VASANTH P – 24619135

LOGA PRAKASH M – 24619120

ABSTRACT

DermaAI is a student project that demonstrates an image-based skin-category classification workflow. The project presentation proposes image preprocessing, ResNet-50 deep feature extraction, and Support Vector Machine (SVM) classification. The accompanying Streamlit program accepts a skin image, converts it to RGB, resizes it to 224 × 224 pixels, prepares a NumPy batch, and obtains output scores from a saved TensorFlow/Keras model.

The screenshots supplied for this report show the interface, an uploaded image, and a model result. In the illustrated run, the highest-scoring category is Eczema with a displayed model score of 58.13%. This is a model-generated output, not a confirmed diagnosis or a verified probability of disease.

The report documents system design, implementation, output interpretation, limitations, and possible improvements. It explicitly distinguishes the methodology proposed in the presentation from the inference code provided. The system is intended for educational demonstration only and must not be used to make medical decisions.

- Keywords: skin image classification, deep learning, ResNet-50, SVM, TensorFlow, Streamlit, image processing.

TABLE OF CONTENTS

CHAPTER / SECTION	PAGE
PRELIMINARY PAGES	**2–4**
Abstract	5
List of Figures	5
Abbreviations	5
CHAPTER 1 INTRODUCTION	**7**
1.1 Background and Motivation	8
1.2 Problem Statement	9
1.3 Need for the Project	10
1.4 Objectives	11
CHAPTER 2 LITERATURE SURVEY	**12**
2.1 Transfer Learning and CNNs	13
2.2 ResNet-50	14
2.3 Support Vector Machine	15
2.4 Research Gap	15
CHAPTER 3 SYSTEM ANALYSIS	**16**
3.1 Existing System	17
3.2 Proposed System	18
3.3 Functional Requirements	19
CHAPTER 4 SYSTEM DESIGN AND METHODOLOGY	**20**
4.1 System Architecture	21
4.2 Dataset Description	22
4.3 Image Preprocessing	23
4.4 Data Augmentation and Splitting	—
4.5 ResNet-50 Feature Extraction	24
4.6 SVM Classification	25

TABLE OF CONTENTS (CONTINUED)

CHAPTER / SECTION **PAGE**

4.7 Evaluation Metrics 26
CHAPTER 5 IMPLEMENTATION 27
5.1 Model Loading 28
5.2 Class-Label Management 29
5.3 Image Upload and Preview 30
5.4 Prediction Logic 31
5.5 User Interface 32
5.6 Project Folder Structure 33
CHAPTER 6 RESULTS AND DISCUSSION 34
6.1 Application Home Screenshot 35
6.2 Image Upload Screenshot 36
6.3 Prediction Screenshot 37
6.4 Interpretation of Example Output 38
6.5 Result Summary and Discussion 39
CHAPTER 7 TESTING, LIMITATIONS, AND RESPONSIBLE USE 41
7.1 Functional Test Plan 42
7.2 Model Evaluation Plan 43
7.3 Known Limitations 44
7.4 Responsible Use and Privacy 45
CHAPTER 8 CONCLUSION AND FUTURE ENHANCEMENTS 46
8.1 Future Enhancements 47
REFERENCES 48
APPENDIX A Program Logic ..... —
APPENDIX B Running the Application 49
APPENDIX C Final Verification Checklist 50

CHAPTER 1: INTRODUCTION

Skin-image classification is a computer-vision task that maps a digital image to one of a set of predefined labels. A model learns visual patterns from examples and produces a score for each configured category. Such outputs can support experimentation, but visual similarity between different skin conditions makes reliable classification challenging.

This project combines an image-upload interface with a trained model. The user can select a JPG, JPEG, PNG, or WEBP file, preview the image, and request classification. The application then displays the highest-scoring category and the scores returned for all configured labels.

- The application is an educational prototype, not a diagnostic device.

The workflow is organized around a clear sequence: image selection, standardized preparation, model inference, and presentation of the resulting class scores. Separating these stages makes the application easier to test and helps identify where errors may occur.

Image classification is affected by variation in illumination, focus, background, camera quality, and the appearance of skin across individuals. A model output should therefore be treated as a prediction produced under learned conditions rather than as proof of a medical condition.

The scope of this mini project is the demonstration of a computer-vision interface and its inference path. Establishing clinical usefulness would require separate validation using appropriately labelled data and expert review.

The complete workflow should be understood as an educational prototype. It demonstrates how an image-processing pipeline can be connected to a web interface, but it does not establish clinical suitability. A user should be able to distinguish the uploaded image, the model output, and the cautionary note without ambiguity. The report therefore describes observed application behaviour separately from the proposed training method and from any performance claims that would require a labelled test set.

1.1 Background and Motivation

Skin images vary because of lighting, camera distance, resolution, skin tone, lesion size, and image acquisition method. A useful computational workflow must apply consistent preprocessing and must be evaluated on data that represent the conditions in which it may be used.

Deep convolutional neural networks learn hierarchical image representations. Residual networks such as ResNet-50 use shortcut connections to help train deeper architectures. A separate classifier such as an SVM can be trained on extracted feature vectors, which is the approach described in the presentation.

- Consistency of preprocessing is essential.
- Evaluation must include class-wise results, not accuracy alone.

Consistent image dimensions make it possible to pass inputs through a model with a fixed input shape. RGB conversion also provides a predictable channel format, although it does not by itself ensure that pixel scaling matches the model training process.

A reliable experiment should document the source of each image, the preprocessing operations, the split between training and evaluation data, and the version of the model used. Such records help another person reproduce the experiment and interpret differences between runs.

In practice, reproducibility depends on keeping preprocessing identical during training and inference. Image dimensions, channel order, pixel scaling, and any model-specific normalization all influence the input received by the network. Documentation of these settings is necessary before predictions from different runs can be compared. Differences in image capture conditions should also be considered when interpreting outputs.

1.2 Problem Statement

The project addresses the problem of organizing a skin-image classification workflow that can accept an image and return a category score. The central technical questions concern image preparation, model inference, correct label mapping, and transparent communication of results.

A model may output a high score even when an image is outside its training distribution. Consequently, the user interface must explain that model output is uncertain and cannot replace clinical assessment.

The problem has both software and modelling aspects. The software must handle file selection and display results correctly, while the model must have been trained on representative data and evaluated independently. A working interface alone cannot demonstrate that predictions are accurate.

The output labels and their order are part of the model contract. If a model returns four scores, the application must map those scores to the same four labels in the same order used during training. Incorrect mapping can produce misleading results even when inference completes without an error.

A complete solution must therefore address more than producing a label. It must preserve the relationship between score indices and class names, handle invalid inputs, and communicate uncertainty. Testing should include both normal use and failure cases. Since the supplied screenshot does not provide a verified ground-truth label, it supports only a description of the displayed output, not a claim of correctness.

1.3 Need for the Project

A compact application can make a trained model easier to demonstrate during a project review. It allows an evaluator to observe the input, the processing step, and the output in one place. The project also illustrates how machine-learning inference can be connected to a web interface without requiring the user to write commands.

The educational value lies in reproducible workflow design and critical interpretation of model outputs, rather than a claim that an experimental model is clinically accurate.

- Simple image upload and preview.
- Visible category scores.
- Explicit safety warning.
- A clear separation between prototype output and medical diagnosis.

A single interface reduces friction during demonstrations because the evaluator can inspect the input image and the corresponding output without running separate command-line steps. The same structure can support controlled testing as the model and preprocessing are improved.

The project also provides a practical way to discuss the difference between a software prototype and a validated decision-support system. A responsible demonstration communicates uncertainty and avoids presenting model scores as medical advice.

The prototype also provides a useful foundation for demonstrating how software components interact. The user interface manages the upload and feedback, image-processing libraries prepare the input, and the model returns values for display. Separating these responsibilities makes later maintenance easier. It also allows interface testing to proceed independently from model-quality evaluation.

1.4 Objectives

The project objectives are to develop an image-based classification demonstration, prepare uploaded images consistently, load a trained model, display model output scores, and document the proposed deep-learning methodology.

The broader methodology in the presentation includes dataset preparation, augmentation, ResNet-50 feature extraction, SVM classification, and evaluation with accuracy, precision, recall, F1-score, and a confusion matrix.

- Build a user-friendly interface.
- Check model-output count against class labels.
- Document reproducible setup requirements.
- Avoid unsupported claims about clinical performance.

The objectives can be checked through observable behaviours: the application starts, supported images can be selected, the preview is displayed, inference is triggered, labels correspond to scores, and failure states are communicated clearly.

For the broader proposed methodology, evaluation should use held-out labelled examples and report class-wise measures in addition to an aggregate score. No such test-set metrics are established by the screenshots included in this report.

The objectives should be assessed with evidence. Functional evidence may include a successful launch, preview of supported images, a visible prediction, and clear messages for missing files or mismatched labels. Model-quality objectives require a separate experiment with known labels. Until such an experiment is completed, the report should not assign numerical accuracy or imply clinical effectiveness.

CHAPTER 2: LITERATURE SURVEY

Skin-lesion classification research includes traditional image-processing approaches, hand-crafted descriptors, transfer learning, end-to-end convolutional neural networks, and hybrid systems that combine deep feature extraction with a conventional classifier.

The project presentation summarizes prior work involving AlexNet, Inception-v4, SVM-based approaches, and residual architectures. The observations below follow the presentation's summaries; complete author lists, titles, venues, and DOI details should be checked against the original publications before final academic submission.

The literature relevant to skin-image classification spans hand-crafted image descriptors, classical machine-learning classifiers, convolutional neural networks, transfer learning, and hybrid feature-extraction pipelines. Each approach involves trade-offs in data requirements, computational cost, interpretability, and evaluation needs.

Comparisons between published studies are meaningful only when dataset composition, label definitions, train/test separation, and evaluation metrics are considered. Different datasets or split protocols can produce results that are not directly comparable.

The approaches discussed in this chapter provide context for the proposed pipeline, but summaries alone are not sufficient for a formal literature comparison. Each publication should be checked for its dataset, class definitions, evaluation design, and reported limitations. The supplied presentation is the basis for the topics summarized here; complete citation details should be verified against the original papers before submission.

2.1 Transfer Learning and CNNs

Transfer learning reuses parameters learned from a large image dataset and adapts the network or its features to a smaller target dataset. This can reduce training requirements compared with training a deep network from scratch.

The presentation discusses AlexNet-based approaches and image augmentation. Augmentation can introduce controlled changes such as rotation or flips, but transformations should preserve the medically meaningful label and should not leak augmented copies across training and test splits.

Transfer learning is useful when the target dataset is smaller than the datasets typically used to train large visual models. A pretrained network can provide general visual representations, while fine-tuning may adapt some or all of its parameters to the target task.

Augmentation can improve robustness when transformations resemble plausible variation in image capture. It must be applied after the data split, however, so near-duplicate versions of the same original image do not appear in both training and evaluation partitions.

A fair comparison between methods requires the same data partitions and evaluation measures wherever possible. Augmentation should create plausible training variation without introducing leakage between training and test sets. Validation data can be used for selecting model settings, while the test set should remain untouched until final evaluation. These practices make reported results more meaningful and reproducible.

2.2 ResNet-50

ResNet architectures use residual connections to support optimization in deeper neural networks. In a feature-extraction setup, a pretrained network is used to convert each input image into a numeric representation. A downstream classifier learns a decision boundary over those representations.

The project presentation proposes ResNet-50 for deep feature extraction. The supplied Streamlit program, however, loads a complete .keras model and does not explicitly call a ResNet-50 backbone. The exact saved model architecture must be inspected to establish whether it includes ResNet-50.

Residual connections allow information to pass across blocks of a deep network and help optimization in deeper architectures. In a feature-extraction pipeline, the selected network layer and pooling operation determine the representation supplied to the downstream classifier.

The project presentation proposes ResNet-50, but the supplied application loads a complete Keras model. The saved model must be inspected before stating that ResNet-50 is actually used in the running application.

The name of an architecture should be supported by the actual saved model configuration. If the model is a complete Keras model, its layers and input/output shapes should be inspected rather than assuming the presentation diagram describes the deployed artifact. Recording the model version and preprocessing function also helps ensure that future runs use the same feature representation.

2.3 Support Vector Machine

An SVM is a supervised learning method that finds a separating boundary between classes. Depending on the data, a linear or kernel-based SVM may be used. Feature scaling, class weighting, and hyperparameter selection can influence performance.

In the proposed hybrid pipeline, ResNet-50 features become the SVM inputs. The SVM must be trained and saved with its label mapping, and inference must repeat the same feature-extraction procedure used during training.

An SVM learns a decision boundary from feature vectors. Its performance can depend on the kernel, regularization setting, feature scaling, and class balance. These choices should be selected using training and validation data rather than by repeatedly checking the final test set.

For deployment, the feature extractor and SVM must be saved with the exact preprocessing and label mapping used during training. The current Streamlit source does not load a separate SVM, so this remains part of the proposed design.

The SVM component is only reproducible if its feature input and training configuration are preserved. The saved classifier should be accompanied by the feature-extraction configuration, any scaler, the label order, and the selected hyperparameters. The current app does not load a separate SVM artifact, so these details remain requirements for implementing the proposed hybrid method.

CHAPTER 3: SYSTEM ANALYSIS

System analysis defines the capabilities, constraints, and dependencies of the application. DermaAI depends on a Python environment, the Streamlit interface library, TensorFlow/Keras, NumPy, Pillow, and a compatible trained model file.

The application expects one of two model paths and optionally reads a JSON file containing category names. If the model is missing or cannot be loaded, the program displays an error and stops instead of proceeding with an invalid inference.

The system can be considered in terms of inputs, processing dependencies, outputs, and error conditions. The input is an image file, the processing stage prepares an array and calls the model, and the output stage presents the highest-scoring category together with the available category scores.

Operational correctness and predictive quality are separate concerns. Dependency errors, missing model files, invalid images, or inconsistent labels are software issues; whether a prediction is correct requires labelled evaluation data.

System analysis identifies what the application expects and what it promises to deliver. The model file and label configuration are external dependencies; their absence or incompatibility should be handled explicitly. The visible result should be traceable to the image submitted by the user, and errors should be reported in a way that supports correction rather than encouraging users to interpret a failed run as a valid prediction.

3.1 Existing System

Existing workflows may rely on manual visual inspection, hand-crafted image features, or deep-learning classifiers. Manual review requires trained expertise. Hand-crafted features may be limited in their ability to represent complex visual patterns, while deep models can be sensitive to dataset bias and changes in acquisition conditions.

The project is not intended to replace a clinician. Its purpose is to demonstrate a software pipeline and provide a base for controlled experimentation.

Manual visual assessment relies on professional knowledge and contextual information that an image-only classifier may not possess. Traditional image-processing methods can use colour, texture, and shape features, but performance may be limited when visual patterns are complex or variable.

Deep-learning methods can learn useful representations from data, yet they can also learn dataset-specific artefacts. Evaluation should therefore examine whether results generalize across image sources and relevant population groups.

No single approach removes the need for careful data design and evaluation. Hand-crafted methods can be easier to inspect but may not capture complex visual patterns; deep models can learn richer representations but may depend strongly on the examples used for training. Any comparison should account for dataset differences and should not generalize results beyond the tested setting.

3.2 Proposed System

The proposed system consists of image selection, preview, preprocessing, inference, and result display. In the presentation's proposed training pipeline, image preprocessing is followed by ResNet-50 feature extraction and SVM classification. In the provided app, preprocessing is followed by prediction from a saved Keras model.

These descriptions should be reconciled in the final implementation by verifying the actual model artifact and training code.

The user-facing path begins with selecting an image and reviewing its preview. After the user requests classification, the program prepares the image, obtains model scores, associates them with labels, and displays the result. Each stage can be tested independently.

The presentation describes a ResNet-50 feature extractor followed by an SVM, whereas the supplied application invokes a saved Keras model directly. These descriptions should not be treated as identical until the model artifact and training code are verified.

The user interface and inference model form the deployed path, while data preparation and model fitting belong to the training path. A future complete implementation should document both paths and ensure they share the same image size, normalization, feature extraction, and class mapping. This alignment reduces the risk that a correctly running interface presents scores from an inconsistent pipeline.

3.3 Functional Requirements

The application must provide a file picker for supported image formats, preview the selected image, and provide a clear button for running classification. It must display a result category and model score and provide a comparison of category scores.

The app must detect a missing model and a mismatch between the model output length and the configured class-label count. A warning must remain visible to discourage medical interpretation.

- Supported formats: JPG, JPEG, PNG, WEBP.
- Input size: 224 × 224 pixels.
- Output: category label and model scores.
- Dependency: compatible saved model and matching class labels.

Functional requirements should be observable and testable. For example, a valid JPG should be decoded and previewed; a missing model should produce a clear message; and a model output with an unexpected number of scores should not be silently mapped to an incompatible label list.

Additional robustness checks could include corrupted files, unusually large images, unsupported formats, and exceptions during inference. Such checks improve the usability of the demonstration but do not replace evaluation of the model itself.

Acceptance tests should state the expected outcome before the test is run. For example, a supported image should be previewed, an absent model should cause a clear error, and a mismatch between the number of model outputs and labels should be detected. Recording actual results makes the test plan auditable and prevents untested capabilities from being described as confirmed features.

CHAPTER 4: SYSTEM DESIGN AND METHODOLOGY

This chapter describes both the running inference flow and the model-training methodology proposed in the presentation. Keeping these layers distinct prevents the report from attributing functions to the app that are not present in the supplied code.

The running application receives a single image and uses an already trained model. Training, data splitting, and quantitative evaluation require separate scripts and artifacts.

The design separates the deployed inference path from the training methodology. The deployed application uses an existing model artifact, while training and evaluation require separate procedures for preparing data, fitting model components, selecting settings, and measuring performance.

This distinction is important for accurate documentation: features described in a presentation are proposed steps unless the supplied source code or saved artifacts confirm that they are implemented.

The methodology must be documented at two levels: what the current application executes and what the proposed training pipeline would execute. The current inference code uses a saved Keras model, while the presentation proposes ResNet-50 feature extraction followed by an SVM. The report keeps this distinction explicit so that the architecture is not overstated.

4.1 System Architecture

The user interacts with the Streamlit frontend. The uploaded file is decoded by Pillow and converted to RGB. The image is resized and converted into a float32 array using NumPy. A batch dimension is added, then TensorFlow/Keras performs inference. The resulting scores are mapped to class names and displayed.

For the proposed ResNet-50 + SVM pipeline, the inference module would instead extract features with the exact trained ResNet-50 configuration and pass those features to the saved SVM classifier.

- Input → RGB conversion → resize → array/batch → model inference → label mapping → results.

The application architecture has a presentation layer, an image-preparation stage, a model-inference stage, and a result-display stage. The interface collects the file and user action; image handling converts the file into the expected representation; inference returns scores; and the display layer associates those scores with labels.

For a hybrid ResNet-50 plus SVM design, the inference path would include the same feature extraction settings used in training and then pass the resulting vector to the saved classifier. Any change to image size, normalization, or selected feature layer can affect the resulting representation.

The boundaries between components can be used to isolate faults. File decoding and resizing can be checked before inference, while output dimensions and label mapping can be checked after prediction. In a hybrid implementation, a further check should verify that the feature vector has the dimensions expected by the saved SVM. Each component should have a documented input and output.

4.2 Dataset Description

The presentation identifies HAM10000 as a primary dermoscopic image dataset with 10,015 images and seven diagnostic categories. The dataset is known for class imbalance, so a single aggregate accuracy value can obscure poor performance on underrepresented classes.

The presentation also mentions ISIC benchmark resources. The exact dataset version, licensing terms, class mapping, and split protocol should be recorded from the dataset actually used. The provided app code does not load these datasets.

The presentation identifies HAM10000 and ISIC resources. Before reproducing an experiment, the exact dataset release, image count, label mapping, licence, and split protocol should be documented. The running Streamlit program does not itself load these datasets.

Class imbalance is an important consideration in skin-image datasets. A model can appear strong on aggregate accuracy while performing poorly for less frequent classes, so class-wise precision, recall, F1-score, and a confusion matrix are useful additions to evaluation.

The dataset version and split protocol are part of the experiment, not incidental administrative details. Image-level duplicates or multiple images of the same lesion can cause leakage if related samples are distributed across partitions. Dataset licences and usage restrictions should also be checked before images are redistributed or included in public demonstrations.

4.3 Image Preprocessing

The program opens the uploaded file with PIL and converts it to RGB. It resizes the image to 224 × 224 pixels, converts it to a NumPy array with float32 values, and expands the dimensions to create a batch containing one image.

The code does not explicitly divide pixel values by 255 or invoke a model-specific preprocessing function. The correct preprocessing depends on the model's training pipeline. If preprocessing differs between training and inference, output quality may be degraded.

RGB conversion ensures that grayscale, palette, or other image encodings are converted to a consistent three-channel representation. Resizing standardizes the dimensions, and conversion to a float32 NumPy array prepares the image for numerical inference.

The supplied code does not explicitly divide pixel values by 255 or call a model-specific preprocessing function. The correct normalization must be determined from the training pipeline; adding an assumed transformation without checking training settings could make inference inconsistent.

Preprocessing must be confirmed from the model training code or artifact documentation. Applying an extra normalization step without evidence can change the numeric range expected by the model, while omitting a required step can also degrade results. The same rule applies to colour channels and resizing method. These choices should be tested and recorded rather than guessed.

4.5 ResNet-50 Feature Extraction

In the proposed approach, a pretrained ResNet-50 network transforms an image into a feature vector. The classification head can be removed or bypassed so that the network's learned representation becomes the input to a separate classifier.

Feature extraction settings, input dimensions, normalization, and the selected network layer should be recorded. The inference code must reproduce these settings exactly.

A feature-extraction pipeline typically removes or bypasses the original classification head and uses an intermediate representation as a numeric feature vector. The selected layer, pooling method, input dimensions, and preprocessing settings should be recorded so that the process is repeatable.

When using a pretrained network, the weights and their source should also be identified. If the network is fine-tuned, the training procedure and resulting model version should be documented separately from the downstream classifier.

For repeatable feature extraction, document the network weights, selected layer, pooling operation, input dimensions, and any preprocessing. Feature vectors generated by different layers or preprocessing settings are not interchangeable. The extraction code should be saved with the experiment so that the downstream classifier receives the same representation during training and deployment.

4.6 SVM Classification

The extracted feature vectors and training labels are used to fit an SVM. Model selection should be conducted on validation data, with appropriate consideration of class imbalance. The selected classifier and any feature-scaling object must be saved for later inference.

The current Streamlit code does not import scikit-learn or load an SVM artifact. Therefore, SVM is part of the presentation's proposed methodology rather than a confirmed step in the supplied app.

The SVM training stage pairs each extracted feature vector with its known class label. Hyperparameters should be selected using training and validation partitions, and any feature scaler must be fitted only on the training data before being reused unchanged for validation, testing, and inference.

The provided application does not import scikit-learn or load an SVM artifact. Therefore, the report records SVM as part of the proposed methodology rather than as a confirmed implementation detail.

Model selection should be carried out without repeatedly consulting the final test set. Class imbalance can be considered through suitable training procedures, but the chosen method and its effect should be reported. During deployment, the exact saved SVM and any scaling transformation should be loaded together to ensure the inference path matches the trained pipeline.

4.7 Evaluation Metrics

Accuracy is the fraction of predictions that are correct. Precision measures the correctness of predictions assigned to a class, while recall measures the fraction of examples of that class identified by the model. F1-score balances precision and recall.

A confusion matrix gives class-by-class counts of correct and incorrect predictions. Macro averages give each class equal weight and can be informative when class frequencies differ substantially.

Precision, recall, and F1-score should be calculated per class as well as in macro or weighted averages where appropriate. The confusion matrix helps identify which categories are most often confused, which is particularly relevant when categories share visual characteristics.

A valid evaluation also needs a documented test-set size, ground-truth labels, data split procedure, and preprocessing configuration. The single screenshot result in this report cannot establish accuracy or any other dataset-level performance metric.

A confusion matrix reveals the distribution of errors between categories, while class-wise precision and recall show whether a model misses particular classes or produces many false positives. Macro-F1 gives equal weight to each class and can be informative when class frequencies differ. These measures should be reported only after predictions have been compared with verified labels.

CHAPTER 5: IMPLEMENTATION

The implementation uses Python and Streamlit for the application, TensorFlow/Keras for inference, NumPy for array operations, and Pillow for image decoding. Model artifacts are expected under the models directory.

The application searches for best_skin_model.keras first and skin_disease_model.keras second. Class labels are loaded from models/class_names.json when that file exists; otherwise, a four-category fallback list is used.

The application is implemented in Python with Streamlit providing the user interface, Pillow decoding images, NumPy preparing arrays, and TensorFlow/Keras executing inference. The trained model is an external artifact, so the program source alone is not sufficient to reproduce a prediction.

For a portable project, dependencies should be listed and tested versions recorded. The expected model filename, label file, input shape, preprocessing assumptions, and command used to start the application should be documented together.

Implementation documentation should let another student recreate the software environment and understand the model dependencies. The source file, requirements list, saved model, and label configuration should be versioned consistently. Any difference between the model used in the demonstration and the one described in the presentation should be resolved or clearly documented before final submission.

5.1 Model Loading

The load_model() function is decorated with st.cache_resource. This allows Streamlit to reuse the loaded model across reruns. The function checks each configured path and returns the first existing model.

If loading raises an exception, the app displays the exception text. If neither model path exists, the app shows a message explaining that a trained .keras file is required and stops execution.

Caching the loaded model avoids repeatedly loading a potentially large artifact on each Streamlit rerun. The model-loading function should still report clear errors when the file is missing, incompatible, or unable to load.

A deployment checklist should verify that the model artifact is present at the expected path and can be loaded in the same environment where the app is started. The actual model architecture cannot be confirmed without inspecting the saved file.

Caching can improve responsiveness, but it should not hide an invalid model configuration. Startup testing should confirm that the selected model file loads successfully and that its expected input and output shapes match the rest of the program. A useful error message should identify the missing or incompatible dependency without implying that a prediction was produced.

5.2 Class-Label Management

The class_names.json file must list labels in exactly the same order as the model output units. A mismatch can cause a score to be associated with the wrong category even if the output array length is correct.

The code checks output count against the number of labels, which catches length mismatches but cannot detect a same-length ordering error. The label file should be generated and versioned with the training pipeline.

Class labels are not merely display text; their order must match the model output units. A length check detects a different number of labels and scores, but it cannot detect a same-length ordering error.

The training workflow should export the label mapping alongside the model and version both artifacts together. A small test using known synthetic outputs or a controlled example can help verify that index-to-label mapping behaves as expected.

The mapping from output index to category should be treated as model metadata. A count check is necessary but not sufficient because a reordered label list can still have the correct length. The most reliable practice is to export the class order from the same training process that produced the model and store both artifacts as one versioned release.

5.3 Image Upload and Preview

The file uploader accepts JPG, JPEG, PNG, and WEBP. When a file is selected, Pillow opens it and converts it to RGB. The interface shows a preview and places the classification controls beside it.

A robust extension could catch corrupted-image exceptions, limit image dimensions and file size, and validate that the upload can be decoded before attempting inference.

The preview allows a user to confirm which file was selected before inference. Converting the uploaded image to RGB creates a consistent image object for downstream processing, while support for common formats reduces friction during demonstration.

Further defensive handling could reject unreadable images, limit upload size, and catch exceptions from the image decoder. These safeguards should return understandable messages rather than exposing confusing tracebacks to the user.

The upload stage should be tested with each supported extension and with files that cannot be decoded. Previewing the image helps the user catch accidental file selection, but it does not validate the clinical content of the image. A production-oriented version would also need limits for file size and dimensions and a clear policy for handling uploaded files.

5.4 Prediction Logic

After the user clicks Run Classification, the app resizes the image to 224 × 224, converts it to float32, and adds a batch dimension. It then calls model.predict() with verbose output disabled and uses the first prediction row.

np.argmax selects the index of the largest score. That index is mapped to a label, and the selected score is multiplied by 100 for display. This assumes the model outputs scores that can sensibly be displayed this way; their interpretation depends on the training configuration.

The program adds a batch dimension because model inference typically expects a batch of images, even when only one image is being classified. The first output row is then used for the selected image, and the index of its largest score is mapped to a class label.

Displaying a score as a percentage assumes the model output has an appropriate interpretation. Unless calibration and output semantics have been verified, the displayed value should be described as a model score rather than a probability that a person has a disease.

The highest score is selected by finding the index of the maximum model output. This operation assumes that the output vector corresponds to the configured class order and that the output values are suitable for the display used by the interface. Calibration and medical interpretation cannot be inferred from the percentage formatting alone.

5.5 User Interface

The interface uses a pale background, a gradient hero banner, a warning panel, and a two-column image/result layout. The result section uses metrics for the top category and score, a bar chart for all categories, and an expandable list of detailed scores.

The screenshots supplied with this report illustrate the landing page, the selected image and classification action, and the resulting score comparison.

The interface combines an explanatory banner, a warning, an upload area, a preview, and a result panel. The category chart makes the relative model outputs easier to compare, while the detailed list provides access to the underlying values.

Accessibility and clarity can be improved with descriptive labels, readable text, clear error messages, and a persistent notice that the output is for educational use. Colour and chart position should not be the only way that results are communicated.

A useful interface makes both the result and its limitations visible. The top category and detailed scores should be labelled clearly, and the educational-use warning should remain easy to find. Errors should not be represented as predictions. The layout should also remain readable on different screen sizes and should not depend solely on colour to communicate meaning.

5.6 Project Folder Structure

A typical project folder contains app.py, requirements.txt, and a models directory. The models directory must contain a compatible .keras model and, preferably, class_names.json. The trained model is not embedded in the Python source.

Install dependencies in a virtual environment and run the application from the project directory using the Streamlit command. Pin tested package versions before sharing the project with another machine.

- DermaAI/app.py
- DermaAI/requirements.txt
- DermaAI/models/best_skin_model.keras
- DermaAI/models/class_names.json

Keeping source code, dependencies, model files, and label configuration in clearly named locations makes the project easier to run on another computer. The model file should not be assumed to exist merely because the application source refers to it.

Before sharing the folder, test the documented setup from a clean environment. This helps reveal missing dependencies, incorrect paths, and assumptions that were present only on the original development machine.

The folder structure should be accompanied by concise setup instructions and a tested dependency list. A clean installation test is especially valuable because local paths or cached packages can conceal missing requirements. Model and label files should be distributed only where licensing and privacy permit, and their versions should be documented.

CHAPTER 6: RESULTS AND DISCUSSION

The following results section is based on the three screenshots supplied by the user. The screenshots show the DermaAI interface and one example inference run. They are evidence of the displayed interface and result for that run, not evidence of overall model accuracy.

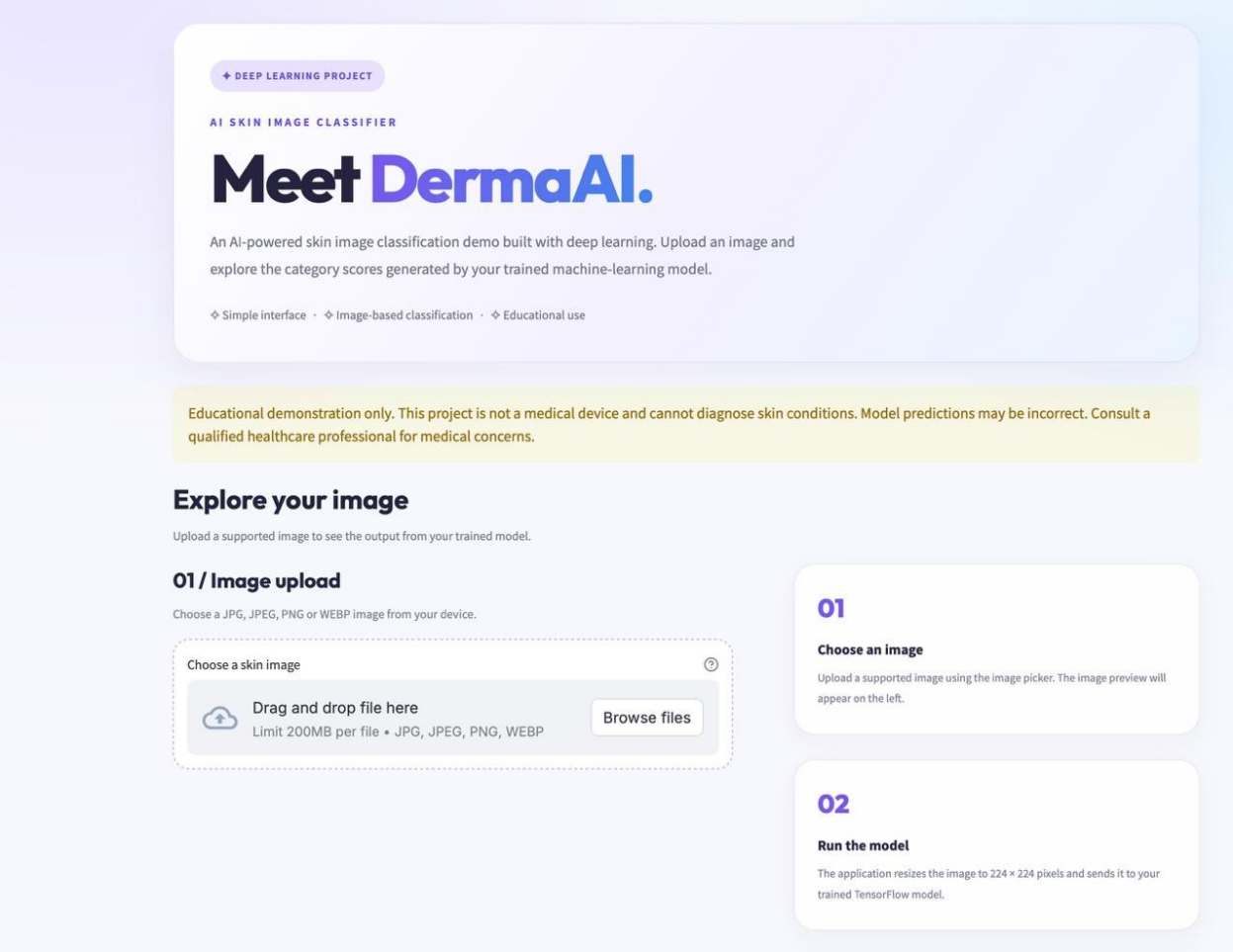
The example image is uploaded as a JPG. The result panel identifies Eczema as the top-scoring model category with a score of 58.13%. The category chart shows four configured labels.

The supplied screenshots demonstrate the application interface and one example inference. They provide evidence that a result was displayed for the selected image, but they do not establish the medical correctness of the label or the model performance on a broader dataset.

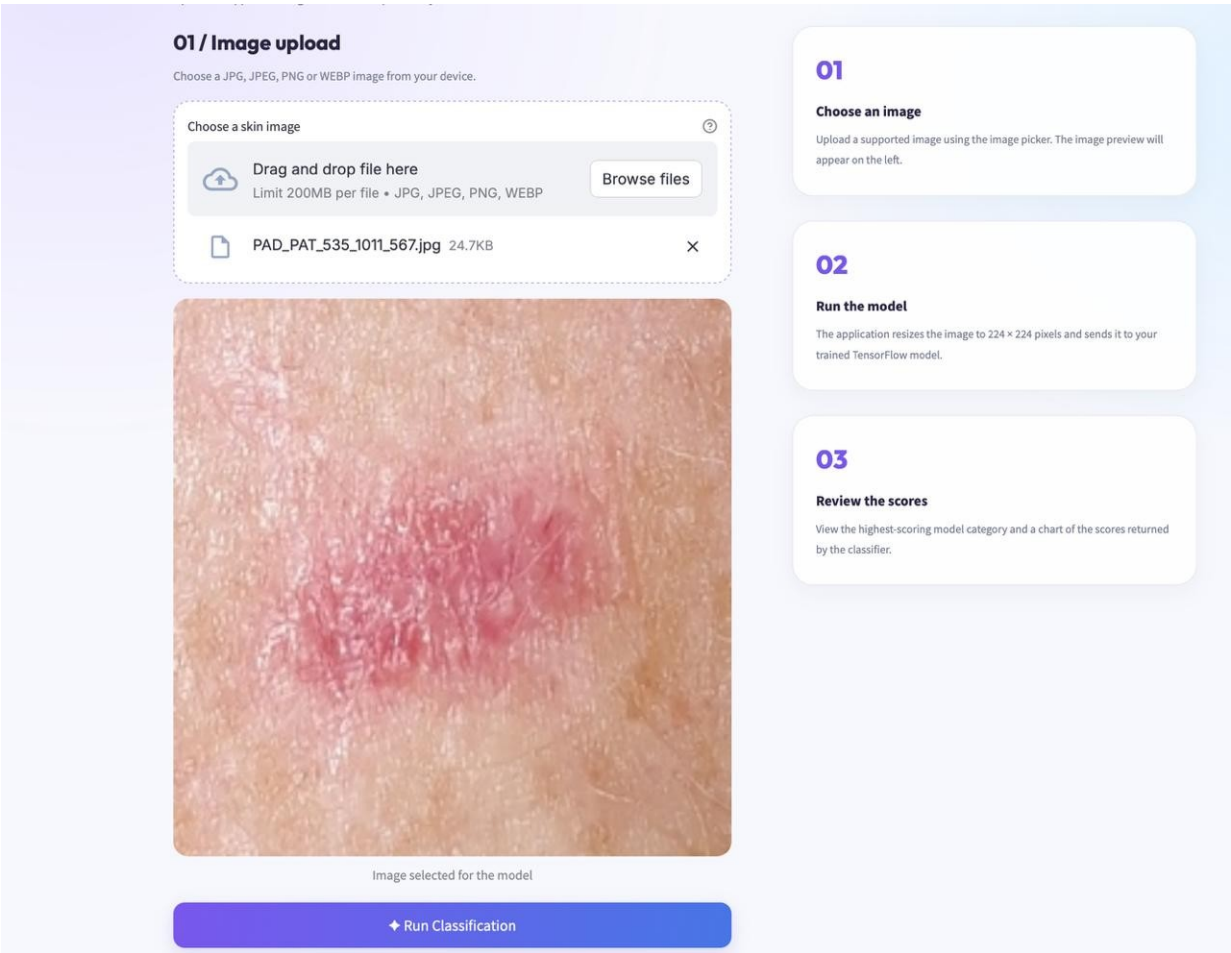
The example output is reported as shown in the interface and is kept separate from quantitative evaluation. This avoids treating a demonstration screenshot as if it were a controlled experiment with a verified ground-truth diagnosis.

The screenshots are qualitative evidence of the interface and the output shown during one run. They do not provide the sample size, reference labels, or test protocol required for performance claims. The results chapter therefore separates observable interface behaviour from conclusions that would require a formal evaluation.

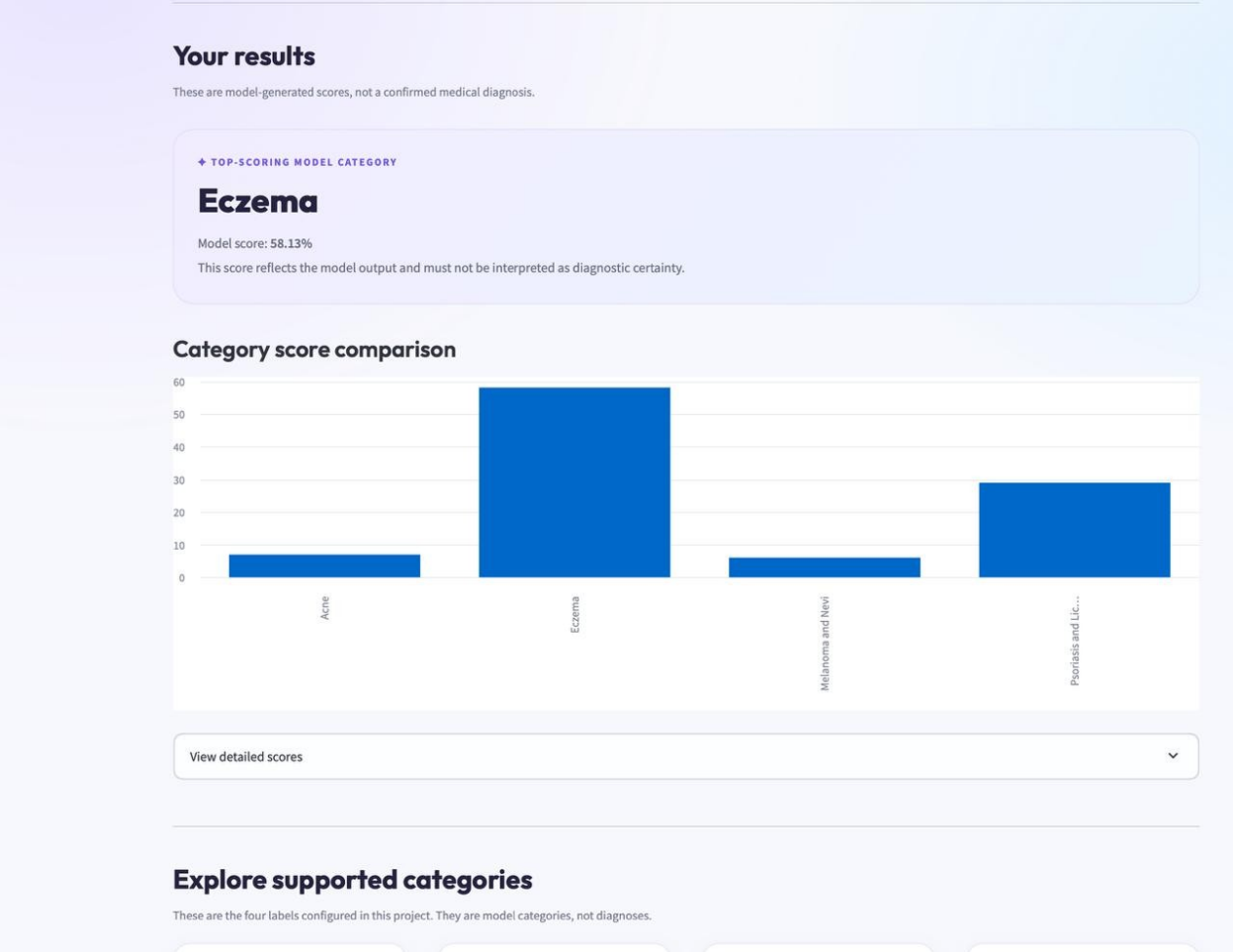
6.1 Result Screenshot: Application Home



6.2 Result Screenshot: Image Upload



6.3 Result Screenshot: Prediction



6.4 Interpretation of the Example Output

For this one illustrated inference, the model's largest output corresponds to the Eczema label. The displayed score is 58.13%. It is not a confirmed diagnosis, and the screenshot alone cannot establish whether the label is medically correct.

The other category scores are visible as bars, but the screenshot does not provide a labelled ground-truth answer for the uploaded image. Consequently, accuracy, precision, recall, F1-score, and correctness for this sample cannot be calculated from the screenshot alone.

Only the top-scoring label and displayed score are established by the supplied screenshot. Without a verified ground-truth label for the same image, the prediction cannot be classified as correct or incorrect.

Performance measures require a collection of examples with known labels and predictions generated under a fixed evaluation protocol. One inference is useful for demonstrating the interface but is insufficient for measuring generalization.

To evaluate the example, a verified reference label would be required for the same image. Without it, the screenshot cannot show whether the predicted category is correct. A broader evaluation should include a sufficiently sized held-out set and should report class-wise errors rather than relying on a single illustrative case.

6.6 Result Summary Table

The table summarizes only what is visible in the supplied screenshots. The example is a single inference run, not a controlled test-set evaluation.

Item	Observed result
Application	DermaAI skin image classifier
Input file	PAD_PAT_535_1011_567.jpg
Top category	Eczema
Displayed model score	58.13%
Configured categories	4
Clinical diagnosis	Not established
Test-set accuracy	Not provided

The summary records the visible application name, uploaded filename, top category, displayed score, and number of configured categories. It deliberately leaves clinical diagnosis and test-set accuracy unestablished because the screenshots do not supply evidence for either.

A future result table should include the test-set definition, total number of evaluated images, class distribution, and class-wise metrics once those values have been measured. Values should not be estimated or invented to make the report appear complete.

The summary is intentionally limited to visible evidence. It records what the interface displayed and marks diagnostic status and test-set accuracy as not established. This avoids presenting missing measurements as though they had been collected. A later version of the table can include quantitative metrics once a documented evaluation has been completed.

6.7 Discussion

The screenshots demonstrate that the upload, preview, run button, and results interface are present. The category chart makes the model output visible and the warning supports responsible interpretation.

A valid performance study would require a defined held-out test set, ground-truth labels, documented preprocessing, and class-wise metrics. The single example cannot be used to claim general performance or clinical effectiveness.

The demonstrated workflow supports the conclusion that the user interface can show an uploaded image and present model outputs. It does not show that the model performs reliably across different devices, skin tones, lighting conditions, or clinical settings.

The next evaluation step is a controlled test using labelled examples not used during model development. Reporting errors and limitations alongside successful examples provides a more balanced account of the system.

The results support a limited conclusion: the interface demonstrates image upload and display of model scores. They do not establish robustness across devices, lighting conditions, skin tones, or patient populations. A useful next step is to test on held-out labelled data and investigate the categories that are most frequently confused.

CHAPTER 7: TESTING, LIMITATIONS, AND RESPONSIBLE USE

Testing should verify that the application accepts supported image formats, handles a missing model, loads label names, performs inference, and detects output-label count mismatches. Model-quality testing is a separate activity requiring labelled data.

The current report records the result shown in the screenshots without extrapolating it to unobserved images.

Testing should cover both ordinary operation and failure conditions. Interface tests can confirm that the application responds correctly to uploads and buttons, while model tests examine the numerical output and its agreement with known labels.

The limitations of the supplied materials include the absence of a verifiable model artifact and the absence of reported test-set metrics. These gaps should be stated explicitly rather than filled with assumed results.

Software testing, model evaluation, and clinical validation answer different questions. Functional tests show whether the application behaves as intended; held-out evaluation measures prediction performance against labels; clinical validation would require additional evidence and appropriate oversight. Success in one area should not be used as a substitute for evidence in another.

7.1 Functional Test Plan

A basic test plan should include normal and error-path cases. Each test should record the environment, input file, expected behavior, actual behavior, and pass/fail status.

Tests should be rerun after changes to dependencies, the saved model, preprocessing, or class labels.

Test	**Expected behavior**
Valid JPG	Image preview appears
PNG/WEBP	Supported image can be opened
Missing model	Clear error and stop
Wrong label count	Mismatch warning shown
Valid model/input	Category scores displayed
Corrupt image	Graceful error handling recommended

Each functional test should record its input, expected result, observed result, and pass/fail status. Tests should include valid image formats, a missing model, a malformed image, a label-count mismatch, and a successful inference using a compatible model.

Retesting is appropriate after changes to dependencies, the model artifact, preprocessing, or class labels. A repeatable test checklist helps detect regressions when the application is moved to another machine.

Test results should be recorded rather than inferred from the existence of code. Each test case should include the environment, input, expected behaviour, observed behaviour, and status. Tests for error handling are important because a missing model or corrupted image should not lead to a misleading result or an unexplained crash.

7.2 Model Evaluation Plan

For a quantitative evaluation, split the labelled data into training, validation, and test partitions before augmentation. Select model settings using training and validation data, and use the test set only for final reporting.

Report the test-set size, class distribution, confusion matrix, per-class precision and recall, macro-F1, and any uncertainty estimates that are appropriate. State the image source and preprocessing so the result can be reproduced.

The data should be split into training, validation, and test partitions before augmentation. Model settings should be selected using training and validation data, while the test partition is reserved for final reporting to reduce optimistic estimates.

The evaluation report should include class distribution, confusion matrix, per-class precision and recall, macro-F1, and a clear description of the data source and preprocessing. The report should state the actual measured values only after the evaluation has been run.

The evaluation protocol should prevent information from the test set influencing model selection. Splitting data before augmentation and keeping related images together helps reduce leakage. The final report should state the dataset version, partition sizes, class counts, preprocessing, metrics, and any important limitations of the experiment.

7.3 Known Limitations

The model artifact and class_names.json were not included in the pasted source code, so this report cannot independently verify the model architecture or reproduce the prediction.

The fallback labels represent four broad categories, whereas the presentation describes seven HAM10000 diagnostic categories. The labels must match the exact model output order. The code does not explicitly normalize pixel values or perform model-specific preprocessing.

- No quantitative test-set metrics were supplied.
- The app does not verify that an uploaded image is a skin image.
- No clinical validation is established by the screenshots.
- One example cannot measure generalization.

The saved model and label configuration were not available for independent inspection in the supplied source materials. As a result, the architecture, exact training procedure, and reproducibility of the displayed output cannot be independently verified from the report alone.

The fallback labels described in the source differ in scope from the seven categories associated with HAM10000 in the presentation. The label configuration must be aligned with the model output, and the app should not imply that the fallback list proves the model was trained on those classes.

The absent model artifact prevents independent verification of the saved architecture and exact inference behaviour. The difference between the four fallback labels and the seven categories discussed for HAM10000 also requires careful checking. Until the model and label metadata are confirmed, architecture and performance claims should remain limited.

7.4 Responsible Use and Privacy

Users should not use the output to diagnose a condition, decide whether a lesion is benign, or choose treatment. Medical concerns should be directed to a qualified healthcare professional.

Only images that the user is authorized to process should be uploaded. If the application is hosted, the project owner should understand file retention, access controls, and the privacy implications of the hosting environment.

The output must not be used to diagnose a condition, decide whether a lesion is benign, or choose treatment. Anyone with a medical concern should seek assessment from a qualified healthcare professional rather than relying on the application.

Only images that the user is authorized to process should be uploaded. If the application is hosted, its owner should review access control, storage, retention, and deletion practices so that sensitive images are not retained or shared unexpectedly.

Skin images can contain sensitive personal information. The application owner should limit collection to images that are authorized for processing and explain any retention or hosting practices. Users should be told that the tool is educational and that medical concerns require a qualified professional. No result should be used alone to decide treatment.

CHAPTER 8: CONCLUSION AND FUTURE ENHANCEMENTS

The project demonstrates an educational skin-image classification interface. The supplied program prepares an image, runs a saved Keras model, and presents category scores. The screenshots show a sample run in which Eczema is the highest-scoring category at 58.13%.

The presentation's ResNet-50 + SVM approach should be treated as the proposed methodology unless the actual trained model and inference artifacts confirm that it is the implemented pipeline. A single displayed result is not evidence of model accuracy or clinical validity.

The mini project demonstrates how a web interface can connect image upload and preprocessing to a saved model and present class scores. The example result shows Eczema as the highest-scoring category, but it does not establish medical correctness or overall performance.

The proposed ResNet-50 plus SVM pipeline remains distinct from the confirmed behaviour of the supplied Keras inference code until the saved model and training artifacts are inspected. This distinction is central to accurate technical documentation.

The project provides an educational demonstration of image upload, model inference, and presentation of category scores. The visible Eczema result is one example only. The report does not claim clinical correctness or measured overall accuracy because the materials do not include the evidence needed to support those claims.

8.1 Future Enhancements

Future work can strengthen reproducibility and reliability by packaging the training and inference code together, recording the model architecture and preprocessing, and storing class labels with the model artifact.

The application can be extended with input validation, clearer class descriptions, model version information, evaluation summaries, and accessibility improvements. Any clinical use would require separate validation and appropriate expert and regulatory review.

- Implement the full ResNet-50 + SVM pipeline if it remains the selected design.
- Add a documented evaluation script and held-out test results.
- Confirm preprocessing and class order.
- Evaluate diversity and generalization.
- Improve input validation and privacy safeguards.

Future work can package training and inference code together, version the model with its label mapping, and record preprocessing settings. A repeatable evaluation script can then generate metrics from a held-out test set and produce a clear report of model behaviour.

Other improvements include validation of uploaded files, more descriptive error messages, accessibility testing, model-version display, and stronger privacy controls. Any intended clinical use would require separate clinical validation and appropriate expert and regulatory review.

A stronger release would package training and inference code, model weights, label mapping, preprocessing settings, and dependencies as a versioned unit. Automated tests and a documented evaluation script would improve reproducibility. Usability, accessibility, file validation, and privacy controls can be improved independently of the model architecture.

REFERENCES

1. Li, L.-F., Wang, X., Hu, W.-J., Xiong, N. N., Du, Y.-X., & Li, B.-S. (2020). Deep learning in skin disease image recognition: A review. *IEEE Access, 8*, 208264–208280. https://doi.org/10.1109/ACCESS.2020.3037258

2. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 770–778).

3. Tschandl, P., Rosendahl, C., & Kittler, H. (2018). The HAM10000 dataset: A large collection of multi-source dermatoscopic images of common pigmented skin lesions. *Scientific Data, 5*, Article 180161. https://doi.org/10.1038/sdata.2018.161

4. Brinker, T. J., Hekler, A., Enk, A. H., Klode, J., Hauschild, A., Berking, C., et al. (2019). A convolutional neural network trained with dermoscopic images performed on par with 145 dermatologists in a clinical melanoma image classification task. *European Journal of Cancer, 111*, 148–154. https://doi.org/10.1016/j.ejca.2019.02.005

5. Haenssle, H. A., et al. (2018). Man against machine: Diagnostic performance of a deep learning convolutional neural network for dermoscopic melanoma recognition in comparison to 58 dermatologists. *Annals of Oncology, 29*(8), 1836–1842. https://doi.org/10.1093/annonc/mdy166

6. Tschandl, P., et al. (2019). Expert-level diagnosis of nonpigmented skin cancer by combined convolutional neural networks. *JAMA Dermatology, 155*(1), 58–65. https://doi.org/10.1001/jamadermatol.2018.4378

7. Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems, 25*.

8. Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv:1409.1556. https://arxiv.org/abs/1409.1556

9. Abadi, M., et al. (2016). TensorFlow: Large-scale machine learning on heterogeneous distributed systems. arXiv:1603.04467. https://arxiv.org/abs/1603.04467

10. Keras. (n.d.). Keras applications documentation. https://keras.io/api/applications/

11. International Skin Imaging Collaboration (ISIC). (n.d.). ISIC Archive and challenge resources. https://www.isic-archive.com/

12. Project presentation. (n.d.). *Image processing on skin diseases using deep learning* [Course project presentation].

13. DermaAI project source code. (n.d.). Streamlit application and TensorFlow/Keras inference implementation [Unpublished source code supplied with the project].

Note: The review article above was supplied as a source PDF. The listed works are selected references relevant to skin-image classification, deep learning, datasets, and the implementation tools; verify citation style and any project-specific source details against the original materials before final submission.

APPENDIX B: RUNNING THE APPLICATION

1. Create a Python environment and install Streamlit, TensorFlow, NumPy, and Pillow.
2. Save the supplied program as app.py.
3. Place the compatible trained model in the models directory as best_skin_model.keras or skin_disease_model.keras.
4. Add models/class_names.json with the exact labels in output order.
5. Run: streamlit run app.py.
6. Upload a permitted image and review the output only as an educational model result.

Before starting the app, confirm that the Python environment contains compatible versions of Streamlit, TensorFlow, NumPy, and Pillow. Place the trained model in the expected models directory and ensure the optional class_names.json file matches the model output order.

Run the application from the project directory using the documented Streamlit command. Upload an image that you are authorized to process, review the displayed output, and remember that the result is an educational model output rather than medical advice.

A clean setup should verify package installation, model loading, and label mapping before an image is processed. The application should be launched from the project directory so relative paths resolve as expected. After launch, use a permitted test image and interpret the result only as an educational output.

APPENDIX C: FINAL VERIFICATION CHECKLIST

Before submission, confirm the college's required cover-page wording, names, register numbers, guide details, and department title. Verify all cited publications and dataset references.

Check that the model artifact is available, class labels match the model, preprocessing is consistent, and the example screenshots are captioned correctly. Replace any placeholder signature/date fields as required by the department.

Do not add unverified model accuracy or claim that the screenshot's top category is a medical diagnosis.

Before submission, verify all names, register numbers, guide details, department wording, certificate fields, signature spaces, and table-of-contents page numbers against the institution requirements. Check that all referenced publications and dataset details are accurate.

Also verify the model artifact, label order, preprocessing consistency, and screenshot captions. Do not insert unverified accuracy figures or describe a predicted category as a confirmed diagnosis.

The final review should check document formatting, names, register numbers, signature spaces, chapter headings, and table-of-contents page numbers. Technical checks should confirm the model artifact and labels, preprocessing consistency, and screenshot captions. Any claims about performance must be supported by actual evaluation results.